Week 7 · Lab 1

Review — Functions Properly:

Prototypes, References, Overloading

Prototypes · `void` and `return` · `<cmath>` · parameters and arguments · scope · default arguments · by value and by reference · overloading

Week: 7 — first session (review, led by the TA)
Duration: 2 hours 15 minutes
Format: the TA explains Lecture 6 again, then practice
Tools: computer with CLion; pen for Sections A–F
Prerequisite: Lecture 6; Week 6 labs

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

LO1. Write prototypes above `main` and definitions below, and explain why the compiler needs them.
LO2. Use `void` functions, an early `return`, and the `<cmath>` functions.
LO3. Predict how arguments are matched to parameters and converted at the call; explain the scope of a local variable.
LO4. Write and call functions with default arguments.
LO5. Choose between passing by value and by reference (`&`); return several results through references.
LO6. Write overloaded functions and predict which one a call selects — or why the call is ambiguous.

This is the review session of Week 7: the TA goes through Lecture 6 again (*functions properly: arguments, scope, default arguments, value and reference, overloading*) and solves problems on it together with the class. Week 7 · Lab 2 is the graded practice on the same material.

How this lab is organised

This is a **review** session, **led by the TA**. In **Part I** the TA explains each idea of Lecture 6 at the board with the worked example; after each idea you solve the short **Quick check** at once, and the TA discusses the answer. In **Part II** the TA solves the *first item* of every section A–F on the board with the class and explains the reasoning; you finish the rest yourself. Sections A–F are on *paper*; Sections G–J are done in **CLion**.

Block	Content		Time
–	Introduction and learning outcomes		5 min
PART I — THE TA EXPLAINS, WITH WORKED EXAMPLES			40 min
§1	Declaration (prototype) and definition		5 min
§2	`void` and an early `return`		4 min
§3	Built-in functions: `<cmath>`		3 min
§4	Parameters, arguments and conversions at the call		4 min
§5	Scope		3 min
§6	Default arguments		4 min
§7	Call by value, call by reference		10 min
§8	Overloading		6 min
§9	Common mistakes		1 min
–	*Short break*		5 min
PART II — PRACTICE (first item of A–F with the TA)			80 min
A	Calls and returns: predict the output	*paper*	7 min
B	Value or reference?	*paper*	9 min
C	Which overload is called?	*paper*	6 min
D	Find the mistake	*paper*	6 min
E	Write the prototype	*paper*	6 min
F	True/false and multiple choice	*paper*	5 min
G	Seconds to a clock time (references)	*CLion*	9 min
H	Quadratic equation (two results by reference)	*CLion*	11 min
I	Areas (overloading)	*CLion*	9 min
J	Bank account (functions + menu)	*CLion*	12 min
K	Bonus challenges *(not timed)*		—
–	Final checklist and wrap-up		5 min
Total			**2 h 15 min**

How your work is checked. Nothing is uploaded. Write the answers to Sections A–F on this sheet; do Sections G–J in CLion, one `.cpp` file per task, with **prototypes above `main` and definitions below**. **When you have finished, raise your hand** and show the TA your sheet and each program running.

PART I — The TA explains: theory and worked examples

1 Declaration (prototype) and definition

The compiler reads top to bottom. A call to a function it has not seen yet is an error — unless a **prototype** (the contract alone, ending in `;`) stands above the call.

```
double area(double r);      // prototypes
int maxOf(int a, int b);

int main() {                // the story
    cout << area(2) << endl;
    cout << maxOf(3, 9) << endl;
    return 0;
}

double area(double r) {     // definitions
    return 3.14159 * r * r;
}
int maxOf(int a, int b) {
    return (a > b) ? a : b;
}
```

Declaration = return type, name, parameter types, `;`. **Definition** = the same plus the body. Declare as often as you like, define exactly once.

With prototypes, a reader meets `main` first and sees *what* the program does; the bodies below say *how*. Parameter names in a prototype are optional — keep them, they document the contract.

Quick check 1. With `void tick() { cout << "* "; }`, what does `tick(); cout << "x "; tick(); tick();` print?

2 `void` and an early `return`

A `void` function *does* something instead of computing a value; it is called as a statement of its own. `return;` with no value leaves it early. In a value-returning function, `return x;` hands `x` back *and ends the function* — lines after it in that branch never run.

Quick check 2. `int sign(int x) { if (x > 0) return 1; if (x < 0) return -1; return 0; }` What does `cout << sign(-5) << sign(0) << sign(7);` print?

3 Built-in functions: `<cmath>`

`sqrt(x)` square root	`pow(x, y)` x^y	`fabs(x)` absolute value
`floor(x)` round down	`ceil(x)` round up	`round(x)` nearest, halves away from zero

All return `double`. You call them exactly like your own functions; `#include <cmath>` brings in their declarations.

Quick check 3. `cout << ceil(-2.5) << " " << round(3.5) << " " << fabs(-0.25) << " " << sqrt(2.25);`

4 Parameters, arguments and conversions at the call

Parameters are the variables in the header; **arguments** are the values in the call. They are matched by number and order, and each argument is **converted to the parameter's type** — by the same rules as an assignment.

Quick check 4. With `int half(int x) { return x / 2; }` and `double halfD(double x) { return x / 2; }`: `cout << half(7.9) << " " << halfD(7);`

5 Scope

A parameter or local variable exists only inside its own function, from its declaration to the closing brace. Two functions may use the same name for different variables — they never meet.

Quick check 5. `int g(int n) { n = n * 2; return n; }` and in `main`: `int n = 5; cout << g(n) << " " << n;`

6 Default arguments

```
double price(double base, double rate = 0.12);

cout << price(100);        // 112
cout << price(100, 0.5);   // 150
```

Defaults fill in **from the right**: once a parameter has one, every parameter after it needs one too. Write the default in **one place** — in the prototype if there is one; repeating it on the definition is an error.

Quick check 6. `int f(int a, int b = 10, int c = 100) { return a + b + c; }`
`cout << f(1) << " " << f(1, 2) << " " << f(1, 2, 3);`

7 Call by value, call by reference

```
void swapValues(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

void split(int total, int& h, int& m) {
    h = total / 60;
    m = total % 60;
}
// in main:
int x = 1, y = 2;
swapValues(x, y);       // now 2 1
int hours, minutes;
split(135, hours, minutes);   // 2, 15
```

By value (`int n`) — the default: the function gets a *copy*; the caller is safe. Without the `&`s, `swapValues` swaps the copies and changes nothing.

By reference (`int& n`) — `n` is *another name for the caller's variable*. Use it to change the caller's variable, or to give back **more than one** answer (`return` gives only one).

Traps: a reference needs a variable (`addTen(5)` does not compile); the type must match exactly; the call looks the same either way — read the header.

Quick check 7. `void inc(int& a, int b) { a++; b++; }` and in `main`: `int x = 1, y = 1; inc(x, y); inc(x, y); cout << x << " " << y;`

8 Overloading

Several functions may share a name if their **parameter lists** differ — in number, types or order. The return type alone does *not* distinguish them. At a call, an exact match wins; otherwise one conversion must be clearly better than the others, or the call is **ambiguous** (a compile error). An overload set and default arguments that accept the same call also collide.

```
int area(int side)              { return side * side; }
int area(int width, int height) { return width * height; }
double area(double radius)      { return 3.14159 * radius * radius; }
```

Quick check 8. With `show(int)` printing `i`, `show(double)` printing `d` and `show(char)` printing `c`: `show(3); show(3.0); show('3'); show(3 + 0.5);`

9 Common mistakes

Mistake	Symptom and fix
Call before any declaration	"undeclared identifier". Add a prototype above `main`.
Missing `&`	Compiles; the caller's variable does not change ("the swap that does nothing").
`f(5)` for `void f(int& n)`	Compile error: a reference needs a variable.
Default written twice	"redefinition of default argument". Keep it in the prototype only.
Overloads differing only by return type	Compile error. Change the parameter list or the name.
Printing inside a function that should return	The value cannot be reused; compute in the function, print in `main`.

PART II — Practice

A Calls and returns: predict the output

[7 min] (10 points)

Each fragment shows the function(s) and the lines inside `main`. (2 each) *The TA solves a) on the board.*

a)

```
int f(int x) {
    return x * 3;
}
int g(int x) {
    return f(x) + 1;
}
// in main:
cout << g(2) << " " << f(g(1));
```

b)

```
void p(int n) {
    if (n <= 0) {
        cout << "stop";
        return;
    }
    cout << n << " ";
}
// in main:
p(3);
p(0);
cout << " end";
```

c)

```
double area(double r) {
    return 3.0 * r * r;
}
// in main:
cout << area(2) / 4 << " " << area(1);
```

d)

```
int sumTo(int n) {
    int s = 0;
    for (int i = 1; i <= n; i++) s += i;
    return s;
}
// in main:
int s = 100;
cout << sumTo(4) << " " << s;
```

e)

```
int price(int base, int extra = 5) {
    return base + extra;
}
// in main:
cout << price(10) + price(10, 0);
```

B Value or reference?

[9 min] (12 points)

Write what each fragment prints. (3 each) *The TA solves a) on the board, drawing the boxes for each variable.*

a)

```
void f(int a, int& b) {
    a += 10;
    b += 10;
}
// in main:
int x = 1, y = 1;
f(x, y);
cout << x << " " << y;
```

b)

```
void swapA(int a, int b) {
    int t = a; a = b; b = t;
}
void swapB(int& a, int& b) {
    int t = a; a = b; b = t;
}
// in main:
int x = 1, y = 2;
swapA(x, y);
cout << x << y << " ";
swapB(x, y);
cout << x << y;
```

c)

```
void twice(int& n) {
    n *= 2;
}
// in main:
int k = 3;
twice(k);
twice(k);
cout << k;
```

d)

```
void grow(int& a, int b) {
    a = a + b;
    b = a;
}
// in main:
int x = 2, y = 5;
grow(x, y);
grow(y, x);
cout << x << " " << y;
```

C Which overload is called?

[6 min] (8 points)

Given `void show(int x) { cout << "int "; }` `void show(double x) { cout << "double "; }` `void show(int x, int y) { cout << "two "; }` — what does each call print, or why does it not compile? (2 each) *The TA solves a) on the board.*

a) `show(7.0);` _____

b) `show('A');` _____

c) `show(3, 4);` _____

d) `long k = 5; show(k);` _____

D Find the mistake

[6 min] (8 points)

Each fragment has exactly one fault. Name it, say whether it is a compile error or a logic error, and fix it. (2 each) *The TA solves a) on the board.*

a)

```
int main() {
    cout << cube(3);
}
int cube(int x) { return x * x * x; }
```

b)

```
void addTen(int& n) { n += 10; }
// in main:
addTen(5);
```

c)

```
double price(double base, double rate = 0.12);
// ... main ...
double price(double base, double rate = 0.12) {
    return base * (1 + rate);
}
```

d) Intended: after the call, `a` and `b` are in increasing order.

```
void order(int a, int b) {
    if (a > b) { int t = a; a = b; b = t; }
}
```

E Write the prototype

[6 min] (8 points)

Write only the **prototype** (header and `;`) of each function. Choose good names and types. (2 each) *The TA solves a) on the board.*

a) Receives a number of seconds and gives back hours, minutes and seconds.

b) Tells whether a year is a leap year.

c) Prints a line of *n* characters; the character is `'-'` unless the caller gives another.

d) Two functions called `maxOf`: one for two `int`s, one for three.

F True/false and multiple choice

[5 min] (8 points)

Mark **T** or **F**; for every **F** write one line why. Then circle one answer. (1 each) *The TA solves statement 1 with the class.*

1. Two overloads may differ only in their return type. ____

2. A function may have several declarations but only one definition. ____

3. Changing a by-value parameter inside a function changes the caller's variable. ____

4. A function can give back two results through two reference parameters. ____

5. With `int f(int a, int b = 2) { return a * b; }`, `cout << f(f(3));` prints
(a) 6 (b) 9 (c) 12 (d) error

6. Which header lets `f(x)` change `x`?
(a) `void f(int x)` (b) `void f(int& x)` (c) `int f(int x)` (d) `void f()`

7. `round(-2.5)` is
(a) −2 (b) −3 (c) −2.5 (d) 2

8. A prototype ends with
(a) `}` (b) `;` (c) `:` (d) nothing

G Seconds to a clock time

[9 min] (10 points)

Task G — one .cpp file

Write `void toHMS(int total, int& h, int& m, int& s)` and `void printTime(int h, int m, int s)`, which prints `HH:MM:SS` with leading zeros (`setfill('0')`, `setw(2)`). Prototypes above `main`, definitions below. `main` reads a number of seconds since midnight (0–86399) and prints the clock time.
Test inputs: `3725` → `01:02:05`; `0` → _____; `86399` → _____; `45296` → _____.

H Quadratic equation

[11 min] (12 points)

Task H — one .cpp file

Write `int solveQuadratic(double a, double b, double c, double& x1, double& x2)`: it **returns** the number of real roots of $ax^2 + bx + c = 0$ ($a \neq 0$) and gives the roots back through `x1` and `x2` ($D = b^2 - 4ac$; $x = (-b \pm \sqrt{D})/2a$). `main` prints the result with two decimals.
Test inputs: `1 -3 2` → `Two roots: 2.00 and 1.00`; `1 2 1` → _____; `1 0 1` → _____; `2 -7 3` → _____.

I Areas

[9 min] (10 points)

Task I — one .cpp file

Write three **overloads** of `double area(...)`: one parameter — a circle of radius r; two — a rectangle; three — a triangle with sides a, b, c (Heron: $s = (a + b + c)/2$, area $= \sqrt{s(s-a)(s-b)(s-c)}$). `main` reads a shape letter (`c`, `r` or `t`) and its sizes, chooses with a `switch`, and prints the area with two decimals.
Test inputs: `c 1` → `Area: 3.14`; `r 2 3` → _____; `t 3 4 5` → _____; `x 1` → _____.

J Bank account

[12 min] (14 points)

Task J — one .cpp file

Keep a `double balance` in `main`. Write `void deposit(double& balance, double amount)`, `bool withdraw(double& balance, double amount)` (returns `false` and changes nothing if the money is not enough) and `void printBalance(double balance)`. `main` runs a menu in a `do-while` loop with a `switch`: `1 amount` deposit, `2 amount` withdraw (print `Not enough money` when it fails), `3` balance, `0` Bye, anything else `Unknown choice`.
Test the whole sequence: `1 100 2 30 2 100 3 1 0.5 3 9 0` → `Not enough money`, `Balance: 70.00`, `Balance: 70.50`, `Unknown choice`, `Bye`.

K Bonus challenges (up to 10 points, not timed)

1. (4) **Fractions.** Write `int gcd(int a, int b)`, `void reduce(int& num, int& den)` and two overloads of `print`: `print(int whole)` and `print(int num, int den)`. Read a fraction and print it in lowest terms, as a whole number when the denominator becomes 1. Test `6 8`, `10 5`, `-4 6`.

2. (3) **Smallest and largest digit.** Write `void minMaxDigit(int n, int& smallest, int& largest)`. Test `4071`, `7`, `98765`.

3. (3) **Default exponent.** Write `long long power(int base, int exp = 2)` (default in the prototype only) and print `power(5)`, `power(2, 10)` and `power(7, 0)`.

Final checklist [5 min]

- ☐ Your **name** and **group** are on the front page.
- ☐ Sections A–F are answered **on paper**, before you used a computer.
- ☐ Every program has its **prototypes above `main`** and the definitions below.
- ☐ Every `&` in your headers has a reason: the function changes that variable or gives a result through it.
- ☐ Default arguments are written once, in the prototype.
- ☐ Your overloads differ in their parameter lists, not only in the return type.
- ☐ Each program was run with **every** test input in its task.
- ☐ You **raised your hand** and showed the TA this sheet and each program running.

Section	Available	Earned
A Calls and returns	10	
B Value or reference?	12	
C Which overload is called?	8	
D Find the mistake	8	
E Write the prototype	8	
F True/false and multiple choice	8	
G Seconds to a clock time	10	
H Quadratic equation	12	
I Areas	10	
J Bank account	14	
Total	**100**	
K Bonus challenges	+10	

Checked by TA: _____

What comes next

- **Week 7 · Lab 2** is the graded practice: LeetCode problems solved with prototypes, references and helper functions.
- Week 7 is also the revision week before the midterm exam in Week 8.

The habit worth keeping from today: **read the header first** — `f(x)` tells you nothing, `void f(int& x)` tells you that `x` may come back changed.

End of Week 7 · Lab 1